Esta guía didáctica desglosa la arquitectura de Bibliotech para facilitar la enseñanza del ciclo de vida de una petición HTTP, integrando conceptos de JavaScript moderno, el motor de Node.js y la seguridad en el acceso a datos.

https://github.com/tecnobfi/bibliotech

1. Fundamentos del Lenguaje y el Entorno

Antes de trazar el recorrido de los datos, es fundamental que los estudiantes comprendan las herramientas sintácticas y arquitectónicas subyacentes.

Desestructuración de Objetos

La desestructuración es una sintaxis de JavaScript que permite extraer propiedades de un objeto y asignarlas a variables en una sola línea. En Bibliotech lo usamos constantemente en las importaciones:

```
JavaScript
const { verifyToken, isAdmin } = require('../middleware/auth_middleware');
```

En lugar de importar todo el módulo (const auth = require(...)) y luego acceder a auth.verifyToken, extraemos exactamente las funciones que el archivo necesita. Esto mejora la legibilidad, reduce el uso de memoria de variables innecesarias y deja claro qué dependencias tiene el script.

Asincronismo: async / await

JavaScript no espera a que terminen las operaciones lentas (como leer un archivo o consultar una base de datos) para seguir ejecutando el código. Para manejar esto, usamos Promesas.

- async: Transforma una función normal en una que siempre devuelve una Promesa.
- await: Pausa la ejecución de esa función específica hasta que la Promesa se resuelva (o rechace). Esto evita el "Callback Hell" (anidamiento infinito de funciones) y permite leer el código asincrónico como si fuera secuencial y sincrónico.

El Event Loop de Node.js

Node.js se ejecuta en un **único hilo principal (Single Thread)**. Si un usuario sube un PDF pesado y Node.js se quedará esperando procesarlo en ese hilo, los demás usuarios del sistema no podrían ni siquiera iniciar sesión. Para evitarlo, Node.js utiliza el **Event Loop** (apoyado en la biblioteca en C libuv). Cuando el hilo principal encuentra una operación de I/O (entrada/salida, como un await pool.query(...)), delega esa tarea pesada a un "Thread Pool" (grupo de hilos secundarios en segundo plano). El hilo principal queda libre inmediatamente para atender a otro cliente. Cuando la base de datos responde, el Event Loop empuja la respuesta de vuelta al hilo principal para que continúe la ejecución.

2. Seguridad: Prevención de Inyección SQL

La Inyección SQL ocurre cuando concatenamos cadenas de texto directamente en la consulta. Si hacemos query("SELECT * FROM users WHERE username = '" + req.body.username + "'"), un atacante podría enviar ' OR '1'='1 y acceder al sistema.

En los repositorios de Bibliotech, mitigamos esto con dos capas de seguridad absoluta:

1. **Prepared Statements (Consultas Preparadas):** Utilizamos el driver mysql2 que soporta el envío de parámetros mediante interrogantes ?. El motor de base de datos compila la estructura de la consulta *primero*, y luego inserta los valores. Los valores siempre son tratados como datos crudos (strings o enteros), nunca como código ejecutable.

2. **Stored Procedures:** Todo el acceso ocurre mediante CALL sp_find_user_by_username(?). La aplicación Node.js no conoce la estructura de las tablas, solo conoce el nombre del procedimiento y los parámetros que debe enviarle.

3. Flujo I: Autenticación y Autorización (Login)

Este flujo explica cómo el sistema identifica a un usuario y le otorga una credencial para navegar.

Fase Frontend

1. **Captura del Evento:** En auth_front_controller.js, el listener loginForm.addEventListener('submit', ...) detecta el clic. La línea e.preventDefault() es crucial para evitar que el navegador recargue la página.
2. **Extracción de Datos:** Se leen los valores del DOM (document.getElementById('username').value).
3. **Petición Fetch:** Se llama a apiService.request('/auth/login', 'POST', { username, password }). El api_service formatea los datos como un string JSON y envía la petición HTTP al servidor. Queda en await esperando.

Fase Backend

4. **Enrutamiento (server.js y auth_routes.js):** server.js detecta el prefijo /api/auth y deriva el tráfico al router correspondiente. Allí, la ruta /login intercepta el POST.
5. **Escudo Sintáctico (validation_middleware.js):** Antes de llegar al controlador, el middleware validateLogin verifica que los campos no estén vacíos. Si fallan, devuelve un error 400 sin molestar a la base de datos. Si pasan, llama a next().
6. **Lógica Semántica (auth_controller.js):**
    - Llama al repositorio: const user = await userRepository.findByUsername(username).

- Verifica la contraseña comparando el hash: await bcrypt.compare(password, user.password).
- Firma la credencial: jwt.sign(...) genera un token encriptado que contiene el ID y el Rol del usuario.
- Devuelve el JSON: res.json({ token, role }).

Fase de Cierre (Frontend)

7. **Persistencia y Redirección:** El api_service recibe el token. En auth_front_controller.js, se invoca authHelper.saveSession(token, role) que lo guarda en el localStorage del navegador. Finalmente, un if evalúa el rol y altera el window.location.href enviando al usuario a su panel correspondiente.

4. Flujo II: Carga Documental (Subir un PDF)

Este flujo explica el manejo de archivos binarios (multipart) y la validación de sesión.

Fase Frontend

1. **Formulario Multipart:** En books_front_controller.js, al enviar el formulario se instancia un objeto new FormData(). Esto es obligatorio porque JSON no soporta el envío de archivos binarios (archivos físicos).
2. **Inyección del Binario:** formData.append('pdfFile', document.getElementById('pdfFile').files[0]) extrae el archivo crudo del input HTML.
3. **Petición Fetch Especial:** En apiService.request(..., isFormData = true), hay una regla vital: **No se debe setear el Content-Type manualmente**. Al pasar un FormData, el navegador genera automáticamente una cabecera multipart/form-data con un *boundary* (separador criptográfico) necesario para que el backend entienda dónde empieza y termina el archivo. El apiService también inyecta el JWT en la cabecera Authorization.

Fase Backend

4. **Barrera de Seguridad (book_routes.js):** La petición choca primero con router.use(verifyToken). Este middleware abre el JWT, verifica su firma criptográfica y, si es válido, extrae el userId y lo inyecta en la petición (req.user = decoded). Solo entonces permite continuar.
5. **Intercepción de Archivos (multer_config.js):** El middleware de Multer lee el *stream* binario entrante. El fileFilter verifica que el *MIME Type* sea exactamente application/pdf. Si es correcto, el diskStorage lo guarda en el disco duro (carpeta /uploads) y le añade un Timestamp al nombre para evitar colisiones. Multer adjunta los metadatos del archivo recién guardado en el objeto req.file.
6. **Validación de Negocio:** Pasa por validateBookUpload para verificar que la cantidad de páginas sea mayor a cero y que no falten datos.
7. **Persistencia (book_controller.js y book_repository.js):** El controlador extrae el ID del usuario (req.user.id), el nombre del archivo generado por Multer (req.file.filename) y los datos del formulario (req.body). Se lo entrega al repositorio que ejecuta el Stored Procedure sp_create_book para guardar el registro final en MySQL.

5. El Ciclo de Vida del Middleware en Express

Un **middleware** en Express es una función que tiene acceso al objeto de petición (`req`), al objeto de respuesta (`res`) y a la siguiente función en el ciclo de vida de la aplicación, comúnmente denotada como `next`.

Actúan como "estaciones de peaje" o escudos preventivos en el flujo de la aplicación.

- **Entrada:** Reciben la petición HTTP cruda enviada por el cliente.
- **Proceso:** Ejecutan código, evalúan condiciones y pueden mutar el objeto de petición (por ejemplo, el `auth_middleware` lee el JWT e inyecta `req.userId` para que el resto del sistema sepa quién hace la petición).

- **Salida:** Tienen dos caminos posibles. Si todo es correcto, llaman a `next()` para pasar el control al siguiente bloque. Si detectan una anomalía (ej. un usuario no logueado o un dato vacío), cortan la cadena inmediatamente utilizando `res.status().json()`, aplicando el principio de diseño *Fail-Fast* (Falla Rápido).

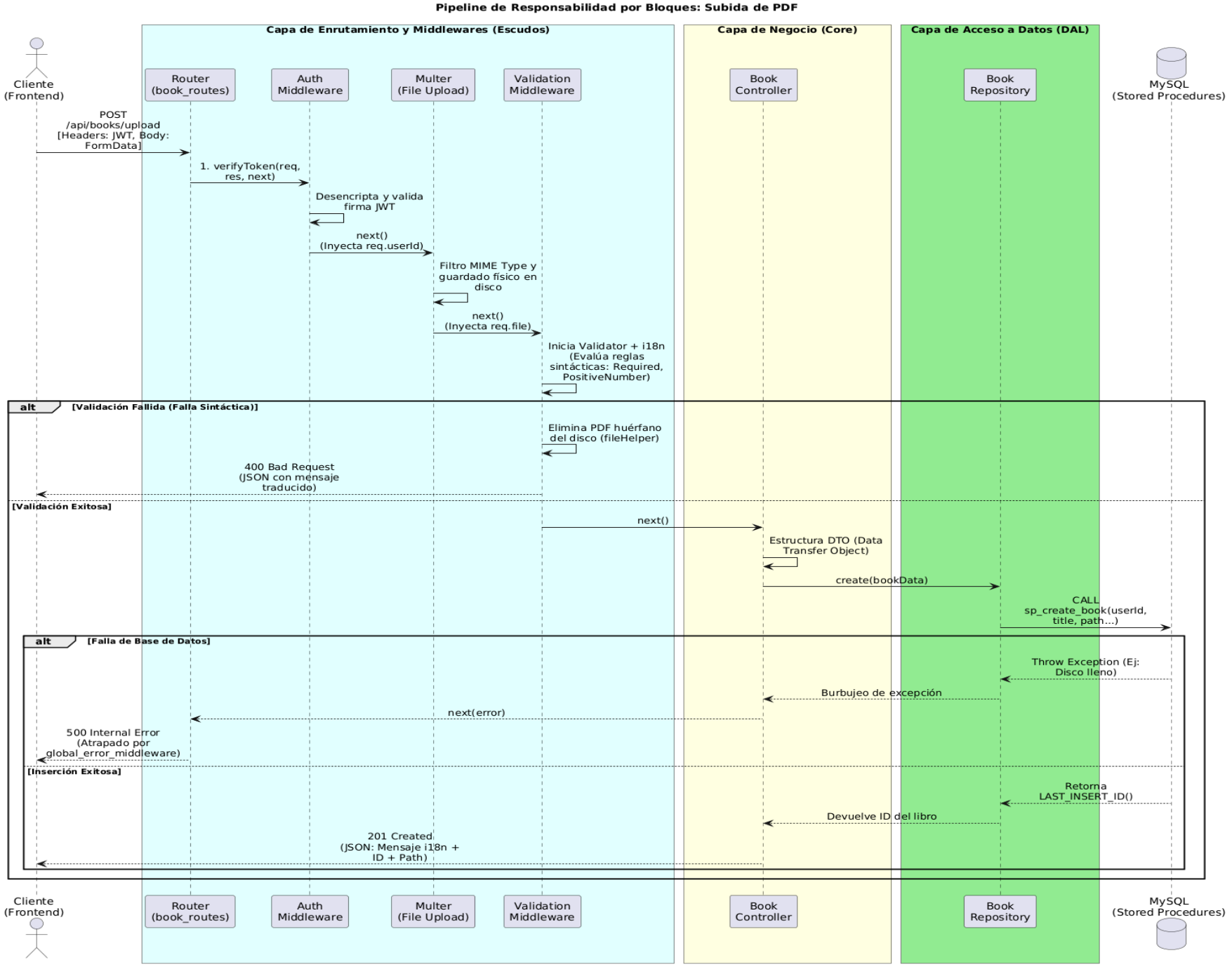
2. Integración Arquitectónica: `Validator` e `i18n`

En arquitecturas limpias, no ensuciaremos los Controladores con docenas de condicionales `if (username === '')`. Delegamos esta responsabilidad al `validation_middleware`, que orquesta dos módulos de utilidad colaborando entre sí.

El Patrón Fluent Interface y Strategy: El `Validator` permite encadenar reglas de validación de forma declarativa e imperativa:`validator.validateThis(password).with(new Required("CT_PASS")).with(new MinLength(6, "CT_SHORT"));` Cada regla (`Required`, `MinLength`) es una Estrategia independiente (Patrón *Strategy*). Esto respeta el Principio Abierto/Cerrado (OCP): podemos crear cientos de reglas nuevas sin modificar jamás el código de la clase `Validator`.

El Patrón Visitor y la Traducción (i18n): El módulo `i18n` es un *Singleton* (una única instancia compartida en toda la aplicación que mantiene el diccionario `es.json` en memoria RAM). Cuando ejecutamos la función `.with(rule)`, el `Validator` se inyecta a sí mismo dentro de la regla (Patrón *Visitor*). Si la regla fracasa, esta utiliza el `i18n` inyectado para buscar la constante `"CT_SHORT"`, recuperar el texto en el idioma activo ("La contraseña es muy corta") y lanzar una excepción exacta que el middleware captura y envía al cliente.

3. Diagrama de Responsabilidad por Bloques (Para visualizar la separación estricta de responsabilidades. Este diagrama de secuencia detalla el ciclo completo de la subida de un documento PDF.)


Pipeline de Responsabilidad por Bloques: Subida de PDF
Capa de Enrutamiento y Middlewares (Escudos)
Capa de Negocio (Core)
Capa de Acceso a Datos (DAL)
Cliente (Frontend)
Router (book_routes)
Auth Middleware
Multer (File Upload)
Validation Middleware
Book Controller
Book Repository
MySQL (Stored Procedures)
POST /api/books/upload [Headers: JWT, Body: FormData]
1. verifyToken(req, res, next)
Desencripta y valida firma JWT
next() (Inyecta req.userId)
Filtro MIME Type y guardado físico en disco
next() (Inyecta req.file)
Inicia Validator + i18n (Evalúa reglas sintácticas: Required, PositiveNumber)
alt [Validación Fallida (Falla Sintáctica)]
Elimina PDF huérfano del disco (fileHelper)
400 Bad Request (JSON con mensaje traducido)
[Validación Exitosa]
next()
Estructura DTO (Data Transfer Object)
create(bookData)
CALL sp_create_book(userId, title, path...)
alt [Falla de Base de Datos]
Throw Exception (Ej: Disco lleno)
Burbujeo de excepción
next(error)
500 Internal Error (Atrapado por global_error_middleware)
[Inserción Exitosa]
Retorna LAST_INSERT_ID()
Devuelve ID del libro
201 Created (JSON: Mensaje i18n + ID + Path)

Análisis del recorrido por capas:

1. **Enrutador (Router):** Recibe la petición bruta. Su única responsabilidad es conocer el mapa de la aplicación y derivar el tráfico al *Pipeline* (tubería) de middlewares correcto.
2. **Middlewares:** Procesan en cadena. Auth responde *"¿Quién es?"*. Multer responde *"¿Dónde guardo el binario?"*. Validation responde *"¿El formulario tiene sentido lógico?"*. Si cualquiera falla, la petición rebota al cliente y nunca llega a consumir recursos del núcleo.
3. **Controlador:** Maneja la semántica. Extrae los datos saneados por los middlewares, orquesta la operación y da la orden de guardar.
4. **Repositorio:** Ignora de dónde vino el dato. Su única función es hablar en lenguaje SQL (mediante parámetros ? blindados contra inyección) con el motor de base de datos.